STUDY GUIDE

Mastering Gradient Descent for Linear Regression

Andrew Ng ML Specialization — Course 1

---

1. Introduction to Gradient Descent

Gradient Descent (GD) is a fundamental optimization algorithm used throughout machine learning. It is the primary method for training linear regression models, and it is also the essential building block behind the most advanced neural networks (deep learning).

Primary Objective

Find the specific values for parameters **w** and **b** that minimize the cost function J(w, b), so the model's predictions get as close as possible to the actual training data.

Generalizability

While we often start with two parameters (w and b), gradient descent scales to functions with n parameters (w_1, w_2, …, w_n and b) — finding the lowest cost across any number of features.

2. Conceptual Intuition: The 'Hill-Walking' Analogy

Imagine standing in a hilly park or on a golf course:

- **The Surface:** the height of the ground represents the cost J. High elevation = high cost, valleys = low cost.
- **Steepest Descent:** spin 360°, find the direction that goes downhill most sharply, and take a 'baby step' that way — the direction of steepest descent.
- **The Iterative Process:** after each step, scan again and take another step, repeating until you can't go further downhill.
- **Local Minima:** in complex functions (e.g. neural networks), your starting point matters — different starts can land in different valleys. Linear regression avoids this problem (see Section 7).

3. The Mathematical Update Rule

Applied repeatedly until the algorithm reaches the minimum:

$$w = w - \alpha \cdot \partial J(w,b)/\partial w$$
$$b = b - \alpha \cdot \partial J(w,b)/\partial b$$

Key Components

$$w = w - \alpha \frac{\partial J(w,b)}{\partial w}$$

learning rate

$$b = b - \alpha \frac{\partial J(w,b)}{\partial b}$$

Simultaneously

- **Assignment operator (=):** in code this means assignment, not equality testing. 'a = a + 1' computes a+1 and stores it back in a — different from '==', which tests equality.
- **Learning rate (α):** a small positive number (typically between 0 and 1) controlling step size.
- **Partial derivative ($\partial J/\partial w$):** determines the direction of the step — which way is 'downhill,' and how big the update should be based on slope steepness.

4. Critical Implementation: Simultaneous Updates

All parameters must be updated **simultaneously**: calculate proposed updates for every parameter using the *original* values before changing any of them.

Correct (Simultaneous)	Incorrect (Sequential)
temp_w = w – α · deriv_w	temp_w = w – α · deriv_w
temp_b = b – α · deriv_b	w = temp_w *(w updated now)*
w = temp_w	temp_b = b – α · deriv_b *(uses NEW w!)*
b = temp_b	b = temp_b

Why 'Incorrect' fails: w gets updated before b's update is calculated. Since b's derivative depends on the model's current state, using a 'poisoned' new w changes the cost function mid-step — producing a different algorithm entirely, not true gradient descent.

5. Selecting the Learning Rate (α)

Scenario	Resulting Behaviour
Too Small	Minuscule steps — eventually reaches the minimum, but painfully slow, needing many iterations.
Too Large	Giant steps overshoot the minimum — cost can increase each step, causing divergence or failure to converge.

Fixed learning rate still converges: you don't need to shrink α over time. As the algorithm nears a minimum, the slope naturally gets smaller — and since the step size is α × slope, steps shrink automatically for a smooth landing.

6. Derivative Intuition and Directionality

The sign of the derivative dictates which way w moves:

- **Positive slope:** tangent line points up-right → derivative positive. w – α(positive) decreases w, moving it left toward the minimum.

- **Negative slope:** tangent line points down-right → derivative negative. w – α(negative) is equivalent to adding, so w increases, moving right toward the minimum.
- **Convergence:** at the minimum, slope = 0, so w = w – α(0) leaves w unchanged. This is convergence.

7. Gradient Descent for Linear Regression

Applying gradient descent to the Squared Error Cost Function, using the linear model $f_{w,b}(x) = wx + b$, the '2' from the squared term cancels with the '2' in 1/2m, giving these neat update rules:

$$\partial J/\partial w = (1/m) \sum (f_{w,b}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$
$$\partial J/\partial b = (1/m) \sum (f_{w,b}(x^{(i)}) - y^{(i)})$$

Convex Functions: The 'Bowl' Shape

Complex models can have many local minima, but the Squared Error Cost Function is **convex** — shaped like a single bowl with exactly one global minimum. So no matter where w and b start, gradient descent always reaches the same optimal point, given an appropriate learning rate.

Batch Gradient Descent

This implementation is called **Batch Gradient Descent** — 'batch' meaning it uses the entire batch of training examples (m) to calculate the derivatives for every single update step.

8. Summary Checklist for Implementation

- ■ Initialize Parameters — set w and b to initial values (commonly w=0, b=0).
- ■ Choose Learning Rate — select an appropriate α to balance speed and stability.
- ■ Calculate Partial Derivatives — use the linear regression formulas for ∂J/∂w and ∂J/∂b.
- ■ Implement Simultaneous Updates — store new values in temp_w and temp_b before updating.
- ■ Verify Convergence — repeat until the change in J(w,b) between iterations is below a small threshold (e.g. 10^{-3}).